

Towards a New Causality Definition

This is a draft, where I describe my ideas regarding the notion of causality and how it should be modified. Here I include the things I had no time to include in the internship report which was prepared for internship defense. Please look into the report for more details.

Made by Andrei Ilin on July 31, 2025.

0.1 The Limitations of Binder's Definition

As shown in the report, the definition of Binder et al. is susceptible to the *gap problem*. Two events e_1 and e_2 can be connected with causality links only if they are connected via ETDG link and there is no event e_3 s.t. $e_3 \rightarrow e_2 \in ETDG$ and $e_3.t > e_1.t$.

But why do we argue that e_1 and e_2 should be causally connected? In the example the pair of traces α and β exhibits only one variation, which is in latency of instruction A . Thus, all the events e s.t. $e^\alpha.t \neq e^\beta.t$ are caused by this variation.

Our goal is to define a new notion of causality that would be able to capture all affected events. Let us start with formulating the expected properties of hypothetical causality definition. Consider two traces α and β where α has a single variation compared to β . We explore a causality links in α .

The definition of causality is expected to have the following properties:

1. **Reachability** — Any event whose timestamp is different between traces α and β must be causally linked to the variation.
2. **Correctness** — If an event is reachable via causal links, it has different timestamps between α and β .
3. **Forwardness** — Given a causality link $e_1 \rightarrow e_2$, e_1 happens earlier than e_2 in trace α .

0.2 Introducing Resolution Procedure

Event-Based Approach

Here we do not take into account branch prediction. Consider single-issue processor.

In our method we focus solely on events in the execution traces. We establish a set of rules which can uniquely define the scheduling inside the processor pipeline.

First, each type of event we define the constraints for timestamp.

1. $\uparrow$ IF — not earlier than $\downarrow$ IF of previous instruction.

2. $\downarrow$ IF — at least 1 cycle must pass since $\uparrow$ IF of the same instruction.
3. $\uparrow$ ID — not earlier than $\downarrow$ ID of previous instruction, not earlier than $\downarrow$ IF of the same instruction.
4. $\downarrow$ ID — at least 1 cycle must pass since $\uparrow$ ID of the same instruction.
5. $\uparrow$ FU — not earlier than $\downarrow$ ID of the same instruction. Cannot be placed between $\uparrow$ FU and $\downarrow$ FU of the same FU by other instruction (exception: can be placed on the same cycle as $\uparrow$ FU of later instruction. The exception depends on scheduling policy). Not earlier than $\downarrow$ FU of instructions from dependencies.
6. $\downarrow$ FU — at least *latency* cycles must pass since $\uparrow$ FU of the same instruction.
7. *COM* — not earlier than $\downarrow$ FU of the same instruction. At least 1 cycle must pass since *COM* of previous instruction.

Second, to uniquely determine the position of event, there should not exist the earlier position where the event can be placed. In other words, each event is scheduled as soon as it is possible by the constraints.

If the rules hold true for an event e , we call e *stable* and *unstable* otherwise. The *stable* set of events is a set where every event is stable.

Proposition 1: For any given input there is only one stable set of events, which corresponds to the set of events derived from execution on the model of the processor.

Step-by-Step Resolution

The idea is that each the rules correspond to the possible dependencies between events (ETDG arcs from Bidner's definition).

Our notion of causality relies on information of both execution traces. The intuition behind our approach is following: if an event e_2 changed its timestamp from trace β to trace α , its movement is either because of variation or the movement of some other event e_1 . In the second case, the causal link $e_1 \rightarrow e_2$ must be established.

Given a set stable set of events E that represents the execution trace, we introduce a variation by changing the latency of one of the instructions. This creates a new set of constraints C . Because of the proposition 1, E is not stable with respect to C , therefore we have a subset of unstable events $U_1 \subset E$. We resolve E by moving all the events form U to the stable position (each to the earliest position where the set of rules hold). However, this creates a set of new unstable events $U_2 \subset E$. We resolve them and repeat the procedure until $U_n = \emptyset$.

Proposition 2: The resolution procedure always terminates.

Proposition 3: The procedure resolves in trace with corresponding to the execution trace of modified input.

Proposition 3 directly follows from propositions 1 and 2.

We illustrate how the resolution procedure works on an example. Consider a following input:

	Res	Dep.	Lat.
A:	FU1		4 1
B:	FU2	{A}	4
C:	FU2		4
D:	FU1	{C}	4

It creates a pair of execution traces α and β with latency of A being 4 and 1 in α and β respectively. Figure 0.1 shows the resolution procedure for this input. We start from trace β , from which we get β_1 by changing the latency of A. Then, on each step unstable events are identified (shown in red boxes) and moved to new positions (shown as green boxes). Note that here we synchronize the movement of $\downarrow$ FU to the movement of corresponding $\uparrow$ FU: this is done for simplicity of visualization. Formally, the described procedure must move $\uparrow$ FU first, and only in the next resolution step the $\downarrow$ FU will be moved. The example illustrates how through a number of resolution step trace β turns into α .

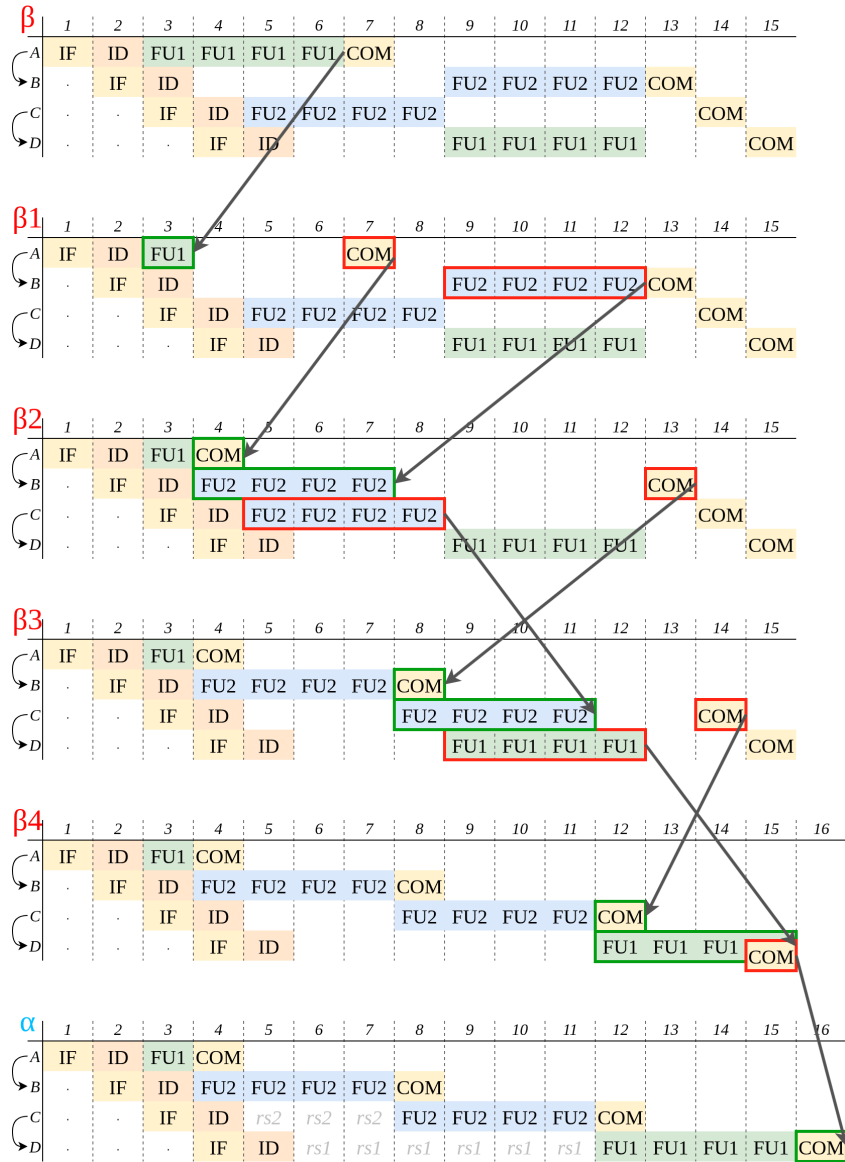


Figure 0.1: An example illustrating the step-by-step resolution procedure

Deriving Causality from Resolution Procedure

The resolution procedure gives us some information about the order of which the events influence each other. We try to define the causality from the resolution procedure as follows:

Consider 3 event sets E_1, E_2, E_3 where the events are different only by timestamps. They represent a part of the resolution procedure ($E_1 \xrightarrow{U_1} E_2 \xrightarrow{U_2} E_3$), where U_1 is a set of events moved in step $E_1 \rightarrow E_2$, U_2 - a set of events moved from $E_2 \rightarrow E_3$. We establish the causality relations between events from U_1 to events from U_2 . To determine which events are responsible for $u \in U_2$ to move, we find the smallest subset $U'_1 \subseteq U_1$ s.t. $E_1 \xrightarrow{U'_1} E'_2 \xrightarrow{U'_2} E'_3$ and $u \in U'_2$. In other words, we search for which events are enough to move in the first step to observe u be moved in the next. Thus, each event may have several incoming multiedges (there can be multiple subsets found). Subsets are disjointed.

Filtering

When applying the procedure some event may be moved several times, so we keep only the arcs corresponding to the last movement of the event.

Figure 0.2 shows the connections restored from example above.

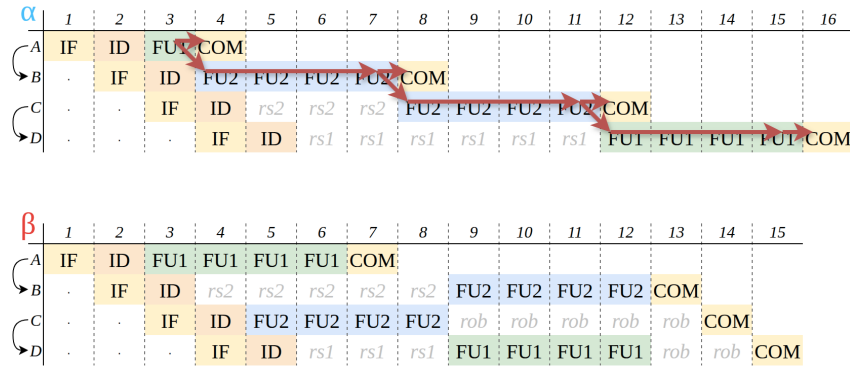


Figure 0.2: Graph restored from resolution procedure shown in figure 0.1

Figure 0.3 shows an example of multiedge: since execution of D is restricted by B and C , both B and C must move to allow D to move.

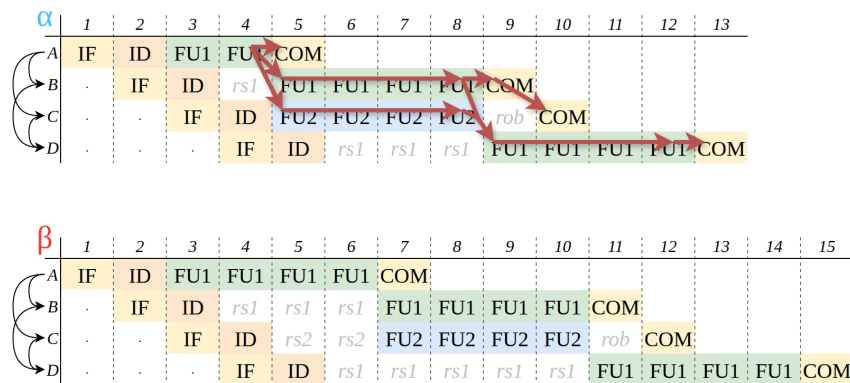


Figure 0.3: Multiedge example

Filtering problem

Example in figure 0.4 is similar to one from figure 0.3. However, here instruction C is split into two: C and D . Normally, we would expect to see a multiedge $(B, D) \rightarrow E$ here. However, due to the filtering we detect the edge $D \rightarrow E$ later than $B \rightarrow E$ and the priority is given to the last discovered.

So the filter should be implemented differently.

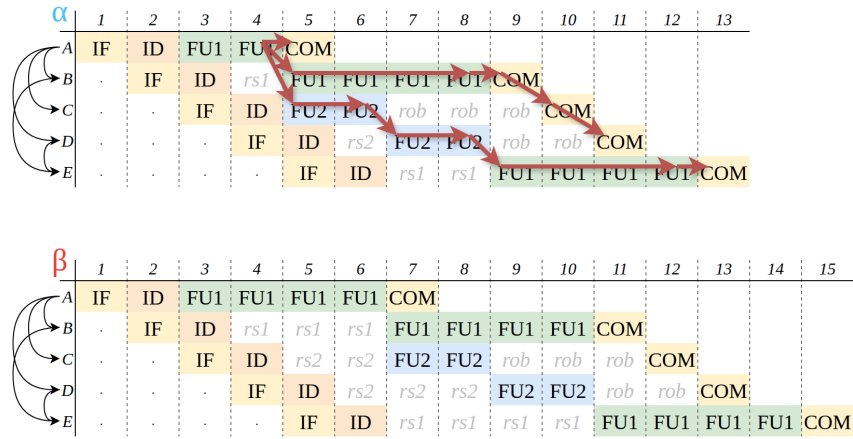


Figure 0.4: Missing edge